ПРАКТИЧЕСКОЕ РУКОВОДСТВО

DeepSeek Harness

Полное руководство пользователя

Версия: 0.2.1-rc.3 · **Платформа:** Windows · **Интерфейс:** английский (русского перевода в этой сборке нет)

> *Это практическое руководство: от первого запуска до тонкой настройки, плагинов, CLI и решения проблем. Названия кнопок и разделов даны так, как они выглядят в интерфейсе, — в скобках или курсивом.*

Версия для печати · включает приложения для лектора

2026

Оглавление

1. Что такое DSH
2. Первый запуск
3. Интерфейс
4. Рабочий цикл: как ставить задачи
5. Разрешения и безопасность
6. Инструменты агента
7. Планирование и контроль
8. Долгие и параллельные задачи
9. Скиллы
10. Настройки
11. Плагины, MCP и пресеты
12. Сессии и данные на диске
13. CLI
14. Рецепты
15. Диагностика
16. Шпаргалка

Приложения

A. Что уточнить начинающему у лектора
B. Вопросы аудитории и готовые ответы

Приложения A и B — в конце документа: чек-лист вопросов для начинающих и готовые ответы для Q&A.

1. Что такое DSH

DeepSeek Harness — это оболочка («харнесс») вокруг языковой модели, которая превращает её в рабочего агента. Модель сама по себе только отвечает текстом; DSH даёт ей:

- **инструменты** — читать и править файлы, искать по проекту, запускать PowerShell-команды, ходить в интернет, создавать документы;
- **сессии** — сохраняемую историю каждого разговора, которую можно продолжить, ответвить или выгрузить;
- **песочницу** — ограничение того, какие файлы агент вправе менять;
- **подтверждения** — остановка перед чувствительными действиями;
- **долгие задачи** — цели, субагенты, фоновые задания, напоминания по расписанию;
- **расширения** — скиллы, плагины, MCP-серверы, пресеты агента.

Одна и та же «начинка» доступна через разные поверхности:

Поверхность	Что это	Когда использовать
Desktop	Приложение Electron с тем же Web GUI внутри	Повседневная работа
Web	`dsh web` — страница в браузере по localhost	Работа из браузера, удалённый доступ
Headless	`dsh headless "задача"` — один запуск без интерфейса	Скрипты, CI, автоматизация
SDK / ACP	Программные интерфейсы для своих клиентов	Интеграции

Все поверхности используют один домашний каталог `%USERPROFILE%\.dsh`, поэтому сессии, настройки и скиллы общие.

2. Первый запуск

2.1. Открыть приложение

Запустите **DeepSeek Harness** из меню «Пуск». Откроется окно с Web GUI; тот же интерфейс доступен в браузере по адресу вида `http://127.0.0.1:51682` (порт меняется при каждом запуске, URL печатается в консоли и хранится в переменной `DSH_WEB_URL`).

Стартовый URL содержит временный токен доступа; после первого открытия браузер получает cookie и остаётся на обычном адресе. Не публикуйте строку запуска с токеном.

2.2. Войти и выбрать модель

1. Откройте **Settings** (шестерёнка внизу боковой панели; горячая клавиша `Mod+,` в Desktop или `Mod+Alt+,` в Web).
2. Раздел **Account** → войдите в **DeepSeek Account**. Это самый простой путь: модели и лимиты подтягиваются автоматически.
3. Альтернатива — раздел **Models**: вставить API-ключ DeepSeek или добавить стороннего провайдера (OpenAI-совместимый, Anthropic-совместимый, свой шлюз).

4. Закройте настройки. В композере (поле ввода внизу чата) нажмите на название модели и выберите модель и **reasoning effort** (усилие рассуждения). В этой сборке по умолчанию стоит `deepseek-flash` через аккаунт DeepSeek с высоким усилием.

Если моделей нет, композер покажет **Select model** / **No models available** — значит, не настроен ни один провайдер.

2.3. Выбрать рабочую папку

Агент работает в **workspace** — каталоге, который является корнем его файловых прав. По умолчанию это папка вида `Documents\deepseek-harness\default-workspace`.

- **Add workspace** в боковой панели → выберите папку проекта.
- Каждая сессия привязана к workspace; сессии группируются по папкам.
- Режим `Workspace Write` разрешает запись **только внутри этой папки** (и во временный каталог).

Для реального проекта лучше сразу добавить его папку, а не работать в `default-workspace`.

2.4. Первый запрос

Напишите задачу в композере и нажмите **Enter**. Например:

> *Посмотри, что лежит в этой папке, и коротко объясни структуру проекта.*

Агент начнёт вызывать инструменты: списки файлов, чтение, поиск. В чате появятся карточки вызовов, а внизу — список задач (`Todo`), если работа многошаговая. Остановить выполнение можно кнопкой **Stop** или двумя нажатиями **Esc**.

3. Интерфейс

3.1. Боковая панель

Слева расположены:

- **Бренд / New Session** — новая сессия в текущем или последнем workspace.
- **Workspaces и Sessions** — дерево папок и разговоров. Панель можно свернуть в узкую полосу (56 px).
- **Plugins** — установленные плагины, настройки официальных модулей, Discover (маркетплейс).
- **Automation tasks** — напоминания и задачи по расписанию.
- **More** — дополнительные панели, если их добавили плагины.
- **Settings** — внизу, всегда на месте.

Список сессий:

- группировка **Group by** → **WorkSpace** (плоские группы) или **Workspace Tree** (вложенные папки);
- сортировка **Last updated** или **Manual** (перетаскиванием);
- **pin** — закрепить наверху;
- **Show more** — раскрыть следующие пять;
- цветные точки статуса: выполняется, **Waiting for approval**, **Plan awaiting review**, **Waiting for answer**, есть непросмотренный результат;
- часы у сессии — есть активные напоминания.

Меню «...» у сессии: **Rename**, **Fork**, **Archive**, **Copy Session ID**, **Download session log** и др. Архив по умолчанию скрыт; чтобы увидеть, переключите фильтр на **All conversations**.

3.2. Чат

- Сообщения пользователя и ответы агента.
- Карточки инструментов: что прочитано, что запущено, что изменено. Их можно раскрывать.
- **Changed files** в конце хода — список изменённых файлов со счётчиками строк. Клик открывает сравнение (diff) в правой панели.
- **Delivery cards** — файлы, которые агент явно подготовил для вас (через инструмент `present`). Их можно открыть в системе.
- **Todo** — чек-лист текущей работы.
- **Goal** — полоса текущей долгой цели (вторая карточка снизу, после Todo).
- **Queue** — очередь сообщений, отправленных во время работы.
- Карточки субагентов и фоновых заданий.

3.3. Композер

Поле ввода — центр управления:

Элемент	Что делает
+	Меню **Add**: File, Goal, Plan, Feedback; и **Commands**: Compact, Permission, Model, Export
/	Командная строка и список слэш-команд
Модель	Выбор модели и reasoning effort
Permissions	Текущий режим прав (Read Only / Workspace Write / Full access)
+ → File / drag-and-drop	Изображения и файлы; можно вставлять из буфера
Enter	Отправить (в занятой сессии — Queue или Steer, см. ниже)
Shift+Enter	Перенос строки
Stop	Остановить текущий ход; на пустом поле кнопка отправки превращается в Stop

Пока агент работает, поле не блокируется:

- **Queue** — поставить сообщение в очередь: агент увидит его после текущего хода.
- **Steer** — вмешаться в текущий ход: агент получит сообщение на ближайшем шаге.
- Какой режим срабатывает по Enter, задаётся настройкой **busy Enter**; `Cmd/Ctrl+Enter` использует противоположный.
- В очереди каждое сообщение можно **Edit**, **Remove** или **Steer**.

Остановка: **Esc Esc** (два нажатия подряд, интервал по умолчанию 500 мс) или кнопка Stop. Очередь при этом сохраняется.

3.4. Правая панель

Открывается по клику на файл, изменение или доставленный документ. Вкладки:

- предпросмотр файла (текст, изображение, PDF, Office);
- **changes-review** — diff изменённых за ход файлов: построчно, side-by-side или unified, с переносом строк;
- вкладки, добавленные плагинами.

Кнопки в шапке diff позволяют открыть файл целиком, переключить вид и (в Desktop) открыть его в системном приложении.

3.5. Шапка сессии

Кнопки справа от названия:

- **Jobs** — фоновые задания с живым выводом и двухшаговой кнопкой остановки;
- **пресет агента** (например, `standard`);
- **more actions** — Rename, Fork, Archive, Download session log, Copy ID, Feedback;
- переключатели правой панели.

4. Рабочий цикл: как ставить задачи

4.1. Формулировка

Хорошая задача для DSH содержит три вещи:

1. **Результат** — что должно получиться («таблица в XLSX», «исправленный тест», «отчёт в MD»).
2. **Критерий готовности** — как проверить, что done («`pnpm test` зелёный», «все ссылки открываются»).
3. **Ограничения** — что нельзя менять, какой стек, какие правила проекта.

Плохо: «сделай красиво». Хорошо: «Приведи README к виду: установка, быстрый старт, troubleshooting. Не меняй код. Проверь, что все команды из README реально существуют в package.json».

4.2. Контекст

- **Файлы**: наберите `@` и путь — появится ссылка-чип; агент прочитает файл.
- **Вложения**: изображения (скриншоты ошибок, макеты), PDF, документы, архивы. Перетащите в окно или вставьте.
- **Папка**: скажите «работай в папке `src/api`» — агент учтёт это в поиске.
- **Правила проекта**: файл `AGENTS.md` (или `.agents/`) в корне читается автоматически и задаёт постоянные инструкции.

Если чего-то не хватает, агент задаст вопрос карточкой **ask_user_question** — это штатный способ уточнить выбор, а не ошибка.

4.3. Вмешательство по ходу

- Агент ушёл не туда → **Stop**, затем уточните.
- Почти всё верно, но нужно добавить условие → отправьте **Steer**-сообщение, не останавливая работу.
- Несколько независимых поручений → отправляйте их в очередь, агент обработает по порядку.

4.4. Приёмка результата

В конце хода смотрите:

1. **Ответ** — краткое резюме и ссылки на файлы.
2. **Changed files** — действительно ли изменено то, что нужно.
3. **Diff** — нет ли лишних правок, не переписан ли файл целиком.
4. **Проверки** — агент должен показать, что запускал тесты/линтер, а не просто заявить об успехе.

Просите проверяемые утверждения: «покажи вывод тестов», «перечисли, какие файлы изменены», «почему ты считаешь, что это работает».

5. Разрешения и безопасность

5.1. Три режима

Разрешения в DSH — это **пресет**, который связывает два независимых параметра: режим песочницы (что можно менять) и политику подтверждений (спрашивать ли человека).

Пресет	Песочница	Подтверждения	Для чего
Read Only	`read-only` — запись запрещена	`ask` — спрашивать	Анализ, ревью, чужой код, «только посмотри»
Workspace Write (по умолчанию)	`workspace-write` — запись только в workspace и temp	`ask` — спрашивать	Обычная работа над проектом
Full access	`danger-full-access` — без ограничений	`never` — не спрашивать	Только доверенная машина/контейнер

Переключение:

- **текущая сессия** — элемент **Permissions** в композере или команда `/permission`;
- **по умолчанию для новых сессий** — **Settings** → **General** → **Permission** (на существующие сессии не влияет).

При выборе **Full access** интерфейс требует отдельно подтвердить риск. **Auto review** (если включён экспериментальный модуль) — отдельный режим с бейджем `EXP`, который подключает автоматического ревьюера.

5.2. Что означает workspace-write

- Агент может **читать** файлы в доступных местах, но **писать** — только внутри корня workspace и во временный каталог.
- Попытка записи за пределами даёт маркер `[sandbox: file access denied under workspace-write mode]`.
- Это ограничение **файловых эффектов**, а не полноценная виртуальная машина: процесс живёт в той же ОС. Для недоверенного кода нужен контейнер/VM, а не просто песочница DSH.

5.3. Подтверждения (approvals)

При политике `ask` чувствительное действие останавливается и ждёт вашего решения. В сессии появляется карточка подтверждения, а в списке — статус **Waiting for approval**.

- Подтверждение действует **только на этот запрос**, а не «навсегда».
- Отклонение возвращает агенту отказ; он должен предложить другой путь.
- Если интерфейс недоступен (headless), действует `never`: запрос отклоняется автоматически, и действие падает безопасно.

Частый случай: агент хочет выполнить команду, которая пишет за пределами workspace. Он может **один раз** повторить тот же вызов с более широким режимом и обоснованием — это вызовет запрос подтверждения. Соглашайтесь, только если понимаете команду.

5.4. Практические правила

1. Держите workspace узким: папка проекта, а не весь диск.
2. Для «посмотреть и объяснить» ставьте **Read Only** — это дешевле, чем откатывать правки.
3. **Full access** — только в одноразовой среде или когда вы полностью доверяете задаче.

4. Не вставляйте в чат пароли и токены: они попадут в журнал сессии. Используйте `.env` в workspace или хранилище учётных данных.
5. Проверяйте diff перед коммитом. Агент может быть уверен, а правка — неверной.
6. Архив/удаление workspace не удаляет файлы на диске — удаляется только группировка.

6. Инструменты агента

Агент сам выбирает инструменты, но полезно знать, что именно он умеет и как об этом просить.

Инструмент	Возможности	Как просить
Чтение файлов	чтение текста, изображений, PDF, Office; диапазоны строк	«прочитай `src/app.ts`», «покажи строки 40–80»
Запись и правка	создание файлов, точечные замены, замена строк	«добавь функцию», «исправь опечатку», «перепиши раздел»
Поиск	поиск файлов по маске, поиск по содержимому (regex)	«найди все вызовы `fetchUser`», «где определяется конфиг»
PowerShell	команды, сборка, тесты, git, скрипты; фоновые задания	«запусти тесты», «собери проект», «покажи `git status`»
Фоновые задания	длинные команды без блокировки чата, чтение вывода, остановка	«запусти dev-сервер в фоне и проверь, что порт открыт»
Веб	поиск и загрузка страниц	«найди актуальную документацию по X», «открой эту ссылку»
Вопрос пользователю	выбор варианта, уточнение	возникает само; можно ответить текстом
Todo	план работ и отметки о выполнении	«составь план и веди чек-лист»
Present	явная выдача файлов-результатов	«подготовь итоговый файл и покажи его»
Скиллы	загрузка специализированных инструкций	«используй скилл office-xlsx»
Цели	долгая цель с авто-продолжением	«поставь цель: …»
Субагенты	делегирование независимых частей	«запусти трёх субагентов на аудит»
Workflow	сценарий оркестрации на JS	«сделай workflow-аудит по всем файлам»
Расписание	напоминания и регулярные задачи	«напоминай каждый день в 9:00»

На Windows команды выполняются в **PowerShell** (`pwsh`), а не в bash. Пути — Windows-формата; в командах используйте `$env:NAME`.

7. Планирование и контроль

7.1. Todo-лист

Для многошаговой работы агент ведёт чек-лист: пункты появляются в панели над композером, текущий помечен как выполняющийся, завершённые отмечаются. Это удобный индикатор прогресса и способ заметить, что агент забыл шаг. Можно попросить: «составь todo и обновляй его».

7.2. Plan mode

Режим планирования заставляет агента **сначала исследовать и спроектировать**, а код менять только после вашего одобрения.

- Вход: `/plan` или `+ → Plan`; можно сразу с задачей: `/plan добавь кэш в API`.
- Выход без плана: `/plan off`.
- Агент изучает проект, затем присылает план и вызывает карточку ревью.
- Кнопки: **Approve** — выйти из плана и начать реализацию; **Keep planning** — вернуть с замечаниями.
- Состояние плана сохраняется при возобновлении сессии и форке.

Важно: plan mode — это **не** ограничение прав. Инструменты остаются доступны; ограничения даёт песочница. Планирование — про порядок работы, а не про безопасность.

7.3. Цели (Goal)

Цель — один долгий результат, который переживает ходы, перезапуск и возобновление сессии.

- Создать: `/goal <формулировка>` или `+ → Goal`; также агент может создать цель сам по вашей просьбе.
- У сессии **одна** текущая цель.
- Фазы: `active`, `paused`, `blocked`, `complete`.
- Полоса цели над композером: редактировать, пауза, продолжить, очистить.
- Автопродолжение ограничено числом раундов (по умолчанию 256).
- После повторного открытия сессии автопродолжение **снимается** — цель нужно явно продолжить (Resume).

Цель нужна для задач вида «доведи миграцию до конца», «почини все падающие тесты». Для обычного одношагового запроса она не нужна.

7.4. Контекст и компакция

Длинный разговор упирается в окно контекста. DSH сжимает историю: старый участок заменяется сводкой, недавние сообщения остаются, а исходный журнал никуда не удаляется.

- Автоматически — по мере роста контекста.
- Вручную — `/compact` или `+ → Compact`.
- Индикатор заполнения контекста показывает, сколько занято.

Если агент «забыл» начало, это может быть компакция. Для важных деталей лучше держать их в файлах проекта, а не только в чате.

8. Долгие и параллельные задачи

8.1. Субагенты

Субагент — отдельный агент со своей сессией и своим контекстом. Родитель делегирует ему часть работы и получает результат.

- **spawn** — новый субагент с нуля;
- **fork** — копия текущей истории (для продолжения в стороне);
- локальные дети сохраняются как сессии, им можно писать (`send_message`), их можно прерывать и перечислять (`list_agents`);
- глубина по умолчанию — 1 (только прямые дети); одновременно живых детей — до 8 (настраивается).

Когда использовать: несколько **независимых** исследований или правок, которые не делят файлы. Когда не использовать: задача, где каждый шаг зависит от предыдущего, — субагент только добавит накладные расходы.

Формулировка: «Запусти двух субагентов: один проверяет API, второй — тесты. Собери вывод и предложи план».

8.2. Workflow

Workflow — это JS-сценарий, который оркестрирует множество субагентов: `agent()`, `parallel()`, `pipeline()`, `phase()`, `log()`. Подходит для аудита по десяткам файлов, миграций, многоракурсного исследования.

Родительский ход блокируется до завершения сценария и получает один итоговый JSON. Обычно достаточно попросить: «прогони workflow-аудит по всем файлам `src/**/*.ts` и верни таблицу проблем». Для одной-двух делегаций workflow избыточен — хватит субагентов.

8.3. Фоновые задания (Jobs)

Долгая команда (сборка, dev-сервер, тесты) может выполняться в фоне, пока вы продолжаете разговор.

- Запуск: попросите «в фоне» или агент сам решит.
- Список — кнопка **Jobs** в шапке сессии; живой вывод раскрывается.
- Чтение вывода, ожидание, остановка — через инструменты `job_output`, `job_list`, `job_kill`.
- Остановка в UI — двухшаговое нажатие.
- Задания живут в процессе DSH: после перезапуска приложения они исчезают (история при этом остаётся в журнале).

8.4. Напоминания и расписание

Задача привязывается к сессии и приходит в неё сообщением.

- Разовые: через N секунд или в конкретный момент.
- Регулярные: фиксированный интервал, ежедневно, по дням недели, cron.
- Создание: попросите агента («каждый будний день в 9:30 проверяй…») или используйте панель **Automation tasks**.
- Хранятся на хосте и переживают перезапуск; пропущенные срабатывания догоняются один раз.
- У сессии с активными напоминаниями в списке горит часы; архивировать такую сессию нельзя, пока напоминания не остановлены.

9. Скиллы

Скилл — это папка с инструкциями, которые агент загружает по необходимости. В отличие от плагина, скилл не добавляет код, а добавляет **знания и порядок действий**: как оформлять документ, как вести ревью, как работать с конкретным фреймворком.

9.1. Формат

```
.dsh/skills/my-skill/SKILL.md
```

или плоский файл `.dsh/skills/my-skill.md`.

```
---
name: my-skill
description: Кратко, когда применять этот скилл.
whenToUse: Один-два примера ситуаций.
---

# Инструкции

1. Первый шаг.
2. Второй шаг.
```

Обязательны `name` и `description`; `description` — главное, по нему модель решает, когда скилл подходит. Необязательные поля: `whenToUse`, `metadata`, `disable-model-invocation`, `user-invocable`.

9.2. Где искать

Скиллы сканируются в порядке приоритета:

1. `<проект>/.dsh/skills`
2. `<проект>/.agents/skills`
3. дополнительные каталоги из настроек
4. `%USERPROFILE%\.dsh\skills`
5. `%USERPROFILE%\.agents\skills`

Проект определяется по ближайшему каталогу с `.git`. Файлы горячо перезагружаются: новый скилл появляется без перезапуска.

9.3. Использование

- Модель видит каталог скиллов и загружает подходящий сама.
- Пользователь может вызвать явно: «используй скилл `office-xlsx`».
- `disable-model-invocation: true` скрывает скилл от модели, оставляя только ручной вызов; `user-invocable: false` — наоборот.

Не путайте:

Сущность	Что добавляет	Где живёт
Скилл	Инструкции	`SKILL.md` в каталоге скиллов
Плагин	Код, инструменты, UI	npm-пакет в профиле
MCP-сервер	Внешние инструменты	Запись в `cordis.patch.yml`
Пресет	Набор инструментов и персона агента	Конфигурация профиля

10. Настройки

Settings открывается шестерёнкой внизу сайдбара. Разделы:

General

- **Appearance** — тема (светлая/тёмная/системная).
- **Coding Tools** — показывать кодовые возможности: карточку изменённых файлов, diff-обзор. Это презентация, а не права доступа.
- **Permission** — пресет по умолчанию для **новых** сессий.
- **Agent preset** — какой пресет агента использовать по умолчанию.
- **Current version** — версия сборки.

- **Open configuration file** — открыть файл конфигурации профиля (только на localhost).

Models

- Список провайдеров: **DeepSeek Account**, **DeepSeek**, сторонние.
- API-ключ вводится один раз и хранится write-only; обратно не отображается.
- Для каждого провайдера — каталог моделей: ID, имя, контекстное окно, максимальный вывод, типы входа (текст/изображения), уровни reasoning.
- **Add model provider** — добавить известного провайдера или **Custom model API** (OpenAI Chat Completions, OpenAI Responses, Anthropic Messages) со своим URL.
- **Restore default models** — сбросить каталог.

Plugins

Страница плагинов содержит:

- **Official** — официальные модули с настройками:
- **Agent loop** — сколько параллельных инструментов выполнять за шаг;
- **Shell** — таймаут команды и лимит вывода на поток;
- **Subagent** — глубина делегирования, лимит параллельных субагентов, разрешённые модели;
- **Web search** — параметры веб-поиска;
- **Session log** — экспорт журнала.
- **Installed / Inventory** — установленные бандлы, версии, совместимость.
- **Discover** — маркетплейс плагинов (если добавлены индексы).

Account

Вход/выход DeepSeek Account, статус подписки, обновления Desktop-приложения.

Настройки сохраняются в патч профиля `%USERPROFILE%\.dsh\profiles\desktop\cordis.patch.yml` и применяются сразу или после перезапуска — зависит от плагина.

11. Плагины, MCP и пресеты

11.1. Плагины

Плагин — npm-пакет, который добавляет инструменты, панели, провайдеров моделей или команды. Установка:

- через **Plugins** → **Discover** (если настроен индекс маркетплейса);
- через CLI: `dsh plugin --profile <имя> add <пакет>`;
- вручную — прописать бандл в `dsh.profile.bundles` профиля.

При установке проверяется совместимость с версией DSH (`engines.dsh`). Несовместимые версии требуют явного исключения — лучше обновить плагин. После установки может понадобиться перезапуск.

11.2. MCP-серверы

MCP (Model Context Protocol) — способ подключить внешние инструменты: GitHub, базы данных, внутренние API. По умолчанию ни один сервер не включён.

Добавьте запись в `cordis.patch.yml` профиля:

```
- id: mcp-github
  name: '@deepseek-ai/dsh-mcp-client'
  config:
    serverName: github
    transport: stdio
    command: npx
    args: ['-y', '@modelcontextprotocol/server-github']
    env:
      GITHUB_TOKEN: !!js process.env.GITHUB_TOKEN
```

Инструменты появятся с именами `mcp__github__<tool>`. Поддерживаются `stdio` и `streamable-http`; есть переподключение, таймауты и лимит размера инструкций. После правки конфигурации перезапустите DSH.

11.3. Пресеты агента

Пресет — это набор инструментов, персона и ограничения для сессии. В поставке:

Пресет	Назначение
`standard`	Обычная работа: файлы, PowerShell, поиск, веб, субагенты, цели, расписание
`minimal`	Облегчённый: постоянная оболочка, чтение изображений, компакция; без файловых правок
`ptc`	Programmatic tool calling — инструменты через код
`cordis`	Расширенный, с инструментами управления плагинами

Пресет по умолчанию для новых сессий выбирается в **Settings** → **General** → **Agent preset**. Смена пресета у активной сессии может убрать часть инструментов — это нормально.

12. Сессии и данные на диске

12.1. Где что лежит

```
%USERPROFILE%\.dsh\
├── profiles\
│   └── desktop\
│       ├── cordis.yml          # корень профиля (не редактировать)
│       ├── cordis.patch.yml    # ваши настройки и плагины
│       └── package.json        # список бандлов
├── sessions\
│   └── <закодированный-workspace>\
│       └── session-<uuid>\
│           └── session.v4.jsonl.zstd
├── storages\
│   ├── workspace.json          # список workspace
│   └── session_projcache\      # кэш проекций
├── skills\                     # пользовательские скиллы
└── .credentials.yaml           # учётные данные (write-only)
```

- Сессии сжаты (zstd) и версионированы; вручную их лучше не править.
- `workspace.json` хранит проекты и порядок; удаление проекта не удаляет папку и сессии.
- `.credentials.yaml` не редактируйте вручную без необходимости: UI хранит секреты в специальном виде.

12.2. Управление сессиями

Действие	Как
Продолжить	Клик по сессии в сайдбаре — история и контекст восстанавливаются

Действие	Как
Переименовать	«…» → **Rename** (или `Mod+Alt+G`)
Ответвить	«…» → **Fork** — копия до последнего завершённого хода
Архивировать	«…» → **Archive**; при активной работе предложит остановить её
Восстановить	Фильтр **All conversations** → **Unarchive**
Скопировать ID	«…» → **Copy Session ID**
Выгрузить	«…» → **Download session log** или `/export` → ZIP с сессией, субагентами и вложениями
Найти	`Mod+K` — поиск по названиям и содержимому

Архив не удаляет историю. Удалить сессию насовсем через UI нельзя — только архивировать; файлы журнала остаются на диске.

13. CLI

DSH поставляется с CLI. В этой установке он лежит здесь:

```
C:\Users\u108\AppData\Local\Programs\DeepSeek Harness\resources\runtime\cli\bin\dsh.cmd
```

Можно добавить этот каталог в `PATH` или вызывать по полному пути. CLI использует тот же `DSH_HOME`, что и Desktop.

13.1. Режимы

```
dsh --version                     # версия
dsh web                           # Web GUI (то же, что dsh --profile web)
dsh --profile web --port 8080 --no-open
dsh headless "прогнать тесты"     # один запуск, печать ответа
dsh headless --json "..."         # поток событий JSON
dsh headless --session-id <id> "продолжи"
dsh plugin --profile desktop add <пакет>
dsh --dump-config                 # показать итоговое дерево конфигурации
```

13.2. Профили

Профиль — именованный набор плагинов и настроек в `%USERPROFILE%\.dsh\profiles\<имя>`.

- `desktop` — зарезервирован за Desktop-приложением; CLI не должен его загружать.
- `web`, `headless`, `sdk`, `sdk-minimal`, `acp` — создаются автоматически из шаблонов при первом использовании.
- Свой профиль: `dsh --profile myprofile --from-default-profile web` — создать копию шаблона.

13.3. Флаги Web

Флаг	Значение
`--port <n>`	Порт (по умолчанию 3080)
`--host <ip>`	Конкретный локальный IP; wildcard запрещён
`--no-open`	Не открывать браузер
`--public-url <url>`	Внешний адрес за прокси
`--trusted-host <host>`	Дополнительный доверенный Host/Origin
`--tls-cert`, `--tls-key`	HTTPS из вашего сертификата

13.4. Headless и учётные данные

Headless не использует вход, сохранённый в GUI. Ключи берутся из:

1. переменных окружения запуска;
2. `.env` в текущем каталоге;
3. `%USERPROFILE%\.dsh\.env`.

```
$env:DEEPSEEK_API_KEY = "..."
dsh headless "суммируй изменения в git diff"
```

Код выхода `0` — задача завершена, `1` — ошибка или прерывание. `--json` даёт машиночитаемый поток: `session`, `status`, `text`, `thinking`, `tool_call`, `tool_result`, `final`.

14. Рецепты

14.1. Разобраться в незнакомом проекте

> *Read-only. Изучи структуру проекта, найди точку входа, опиши архитектуру и основные потоки данных. Составь список файлов, которые стоит прочитать первыми. Ничего не меняй.*

14.2. Безопасный рефакторинг

> *В workspace-write. Переименуй `getUser` в `fetchUser` во всём проекте, обнови импорты и тесты. Сначала покажи план, затем внеси правки, запусти `pnpm test` и покажи diff.*

14.3. Регулярный отчёт

> *Каждый будний день в 9:30 присылай сводку: `git log --since=yesterday`, открытые TODO и упавшие тесты. Если всё зелёное — одно предложение.*

14.4. Параллельный аудит

> *Запусти трёх субагентов: первый проверяет безопасность (секреты, инъекции), второй — обработку ошибок, третий — тесты. Собери таблицу: файл, проблема, серьёзность, рекомендация.*

14.5. Документ из данных

> *Собери XLSX с тремя листами: «Сводка», «Детали», «Графики». Данные возьми из `data.csv`. Проверь формулы пересчётом и покажи файл через present.*

14.6. Долгая цель

> *Поставь цель: довести покрытие тестами модуля `billing` до 80%. Работай автономно, после каждого шага запускай тесты и обновляй todo. Остановись, если понадобится решение, которое можешь принять только ты.*

15. Диагностика

Симптом	Причина	Что делать
Disconnected в настройках	Потеря связи с хостом	Нажать retry; проверить, что приложение запущено; перезапустить
Select model / No models available	Не настроен провайдер	Settings → Models → войти или добавить ключ
Запрос падает с ошибкой авторизации	Ключ не тот/просрочен	Перевыпустить ключ, обновить в Models
`[sandbox: file access denied]`	Запись вне workspace	Работать в нужной папке, сменить permission или подтвердить эскалацию
Агент не видит файл	Файл вне workspace или не тот путь	Добавить workspace, использовать полный путь, проверить `@`-ссылку
Waiting for approval висит	Ожидает вашего решения	Открыть сессию, подтвердить или отклонить
Сессия не архивируется	Активная работа или напоминания	Остановить работу/удалить напоминания, затем архивировать
Долгая команда «пропала»	Фоновое задание завершилось/умерло	Открыть **Jobs**, прочитать вывод; после перезапуска задания не восстанавливаются
Контекст переполнен	Длинная история	`/compact`, разбить задачу, вынести детали в файлы
Плагин не загружается	Несовместимая версия	Обновить плагин или DSH; проверить `engines.dsh`
Порт занят	Другой процесс/экземпляр	`dsh web --port 8081`
Headless не видит вход из GUI	Так задумано	Задать `DEEPSEEK_API_KEY` в env или `.env`
Странные ошибки после обновления	Старый кэш фронтенда	Обновить страницу; перезапустить приложение

Если проблема в доступе к файлам на Windows, в комплекте есть диагностический скилл `diagnose-windows-sandbox-acl`: он проверяет ACL пути и его родителей и чинит подтверждённые проблемы.

16. Шпаргалка

Команды в композере

Команда	Действие
`/model`	Выбор модели и усилия
`/permission`	Режим прав текущей сессии
`/plan`	Войти в режим планирования
`/plan off`	Выйти из планирования
`/goal`	Создать/показать цель
`/compact`	Сжать историю
`/export`	Скачать журнал сессии (ZIP)

Горячие клавиши

Клавиши	Действие
`Enter`	Отправить
`Shift+Enter`	Новая строка
`Cmd/Ctrl+Enter`	Отправить противоположным режимом (Queue/Steer)
`Esc Esc`	Остановить текущий ход
`Mod+K`	Поиск сессий
`Mod+Alt+G`	Переименовать сессию
`Mod+,` (Desktop) / `Mod+Alt+,` (Web)	Настройки
`Mod+/`	Справка по горячим клавишам

`Mod` — это `Ctrl` на Windows и `Command` на macOS. Комбинации можно переназначить в настройках клавиш; часть из них зависит от платформы.

Ключевые пути

Что	Где
Домашний каталог DSH	`%USERPROFILE%\.dsh`
Настройки профиля	`%USERPROFILE%\.dsh\profiles\desktop\cordis.patch.yml`
Сессии	`%USERPROFILE%\.dsh\sessions\`
Скиллы	`%USERPROFILE%\.dsh\skills\`
CLI	`...\DeepSeek Harness\resources\runtime\cli\bin\dsh.cmd`
Workspace по умолчанию	`Documents\deepseek-harness\default-workspace`

Три правила, которые спасают больше всего

1. **Узкий workspace + Workspace Write + подтверждения** — безопасный дефолт.
2. **Проверяйте diff и вывод проверок**, а не только текст ответа.
3. **Долгую работу оформляйте целью или планом**, а не одним гигантским сообщением.

ДЛЯ ЛЕКТОРА

Приложение А. Что начинающему стоит уточнить у лектора

Это вопросы, которые лучше задать **до** самостоятельной работы, чтобы не узнать о границах задним числом. Их же можно раздать слушателям как чек-лист.

№	Вопрос лектору	Что важно услышать в ответе
1	Какой у меня доступ: свой аккаунт, общий ключ или демо-стенд?	Чьи квоты расходуются и что будет при исчерпании лимита
2	Какая модель и какой reasoning effort используются по умолчанию?	Компромисс скорость/качество/цена; как переключить
3	Где мой workspace и что в него входит?	Границы записи; можно ли подключить реальный проект
4	Какой режим прав включён и кто его меняет?	Read Only / Workspace Write / Full access; политика подтверждений
5	Какие действия агент выполняет без спроса?	Что означает approval и почему он одноразовый
6	Можно ли работать с боевым репозиторием или продом?	Запреты, staging, отдельная ветка, бэкапы
7	Как откатить изменения, если агент ошибся?	Git, diff, бэкапы; что «отмены» одной кнопкой нет
8	Куда попадают мои данные и кто их видит?	Провайдер модели, журналы сессий, телеметрия
9	Что категорически нельзя вставлять в чат?	Секреты, персональные данные, внутренние документы
10	Как сохранить и перенести результат работы?	Файлы в workspace, экспорт сессии ZIP
11	Как продолжить работу завтра?	Сессии, история, контекст, компакция
12	Сколько это стоит и как не разориться?	Токены, выбор модели, размер контекста, субагенты
13	Работает ли это офлайн и в корпоративной сети?	Нужен доступ к API; прокси; локальные модели
14	Можно ли подключать свои скиллы, плагины и MCP?	Политика безопасности и кто разрешает установку
15	Как прервать агента и что будет с очередью?	Stop / Esc Esc, Queue и Steer
16	Как проверить, что результат верный?	Тесты, diff, воспроизводимые проверки
17	Что делать при ошибке доступа к файлам?	Sandbox, ACL, смена workspace или прав
18	Есть ли русская локализация и поддержка русского?	Интерфейс EN, общение и документы — RU
19	Как обновляться и где брать новые версии?	Desktop-обновления, совместимость плагинов
20	К кому идти с проблемой: к лектору, в поддержку или в сообщество?	Канал, логи, что приложить к обращению

ДЛЯ ЛЕКТОРА

Приложение B. Вопросы аудитории и готовые ответы

Как пользоваться этим разделом: вопрос выделен жирным, ответ — обычным текстом. Ответы рассчитаны на 20–40 секунд устной речи; если аудитория просит глубже, переходите к соответствующему разделу руководства.

B.1. Начинающие пользователи

1. Чем DSH отличается от обычного чата с DeepSeek?

Обычный чат отвечает текстом. DSH даёт модели инструменты: читать и править файлы, искать по проекту, запускать PowerShell, ходить в интернет, создавать документы. Плюс сессия, песочница и подтверждения. Это не «чат побольше», а рабочий цикл с доступом к вашей папке.

2. Агент может удалить или испортить мои файлы?

Может — в пределах выданных прав. В режиме Workspace Write он пишет только внутри workspace, в Read Only не пишет вообще. Но удаление внутри workspace или destructive-команда в PowerShell возможны, если вы разрешили. Поэтому: git, бэкап и проверка diff — обязательная страховка. Выбирайте Read Only, когда нужно только посмотреть.

3. Как отменить действие агента?

Единой кнопки «отменить всё» нет. Текущий ход останавливается кнопкой Stop или двумя Esc. Уже сделанные изменения откатываются через git (`git checkout`, `git restore`) или из бэкапа; diff показывает, что именно изменилось. Поэтому перед крупной работой полезно сделать коммит.

4. Почему агент постоянно просит разрешение?

Это политика `ask`: чувствительное действие останавливается и ждёт вашего решения. Подтверждение действует только на один запрос, а не навсегда. В режиме Full access подтверждений нет — и это опаснее. Если подтверждений слишком много, скорее всего, выбран не тот режим прав или задача выходит за пределы workspace.

5. Где сохраняются результаты работы?

Файлы — в workspace, в тех путях, которые агент указал в ответе. В конце хода появляется карточка Changed files со списком изменений; явно подготовленные файлы показываются карточками доставки. Журнал разговора хранится в сессии, его можно выгрузить через `/export`.

6. Как продолжить вчерашнюю работу?

Откройте сессию в сайдбаре — история и контекст восстановятся. Если сессия архивирована, переключите фильтр на All conversations и нажмите Unarchive. Новая сессия не наследует старую историю; для продолжения в стороне есть Fork.

7. Сколько стоит работа и как её контролировать?

Платите провайдеру за токены: вход, выход и reasoning. Стоимость растёт от длинного контекста, больших файлов, субагентов и высокого reasoning effort. Снижать: выбирать более дешёвую модель, уменьшать контекст, использовать `/compact`, не запускать десяток субагентов без нужды. Встроенного «бюджетного лимита» нет — счёт формирует провайдер.

8. Можно ли работать без интернета?

Сам интерфейс и локальные инструменты — да, но модель вызывается по сети, поэтому без доступа к API ответов не будет. Можно подключить локальный OpenAI-совместимый сервер как провайдера, но это отдельная настройка и отдельные требования к качеству модели.

9. Интерфейс на английском — это надолго?

В этой сборке локализация английская (плюс китайская), русского интерфейса нет. На результат это не влияет: задачи можно формулировать по-русски, документы и ответы тоже будут на русском.

10. Как приложить файл или папку?

Изображения и файлы — через `+ → File`, drag-and-drop или вставку из буфера. Ссылку на конкретный файл удобно вставить через `@`. Папку целиком «приложить» нельзя: агент работает с ней через workspace и инструменты поиска.

11. Почему запись за пределы папки запрещена?

Workspace — это граница прав, а не просто «текущая папка». Так агент не может случайно изменить системные файлы или чужой проект. Если нужно писать в другое место, добавьте его как workspace или смените режим прав осознанно.

12. Как понять, что агент не выдумывает?

Просите проверяемые вещи: вывод тестов, список изменённых файлов, diff, конкретные строки кода. Смотрите не на уверенный тон, а на доказательства. Если агент сослался на файл — откройте его. Если сказал «тесты проходят» — попросите показать вывод.

B.2. Опытные пользователи

1. Чем goal отличается от todo и plan mode?

Todo — чек-лист внутри хода. Plan mode — режим «сначала спроектируй, потом делай» с вашим одобрением. Goal — durable-объектив: одна долгая цель, которая переживает ходы и перезапуск, с автопродолжением и лимитом раундов. Их можно комбинировать: цель задаёт «куда», план — «как», todo — «что сейчас».

2. Когда субагенты, а когда workflow?

Субагенты — когда частей две-три и они независимы: каждый получает свой контекст, результат возвращается родителю. Workflow — когда частей десятки и нужен сценарий: `parallel`, `pipeline`, фазы, сбор структурированного результата. Для одной делегации workflow избыточен, для аудита сотни файлов — субагенты вручную неудобны.

3. Что именно теряется при компакции?

Старый участок диалога заменяется сводкой; недавние сообщения остаются. Исходный журнал не удаляется и доступен в экспорте, но модель видит сводку, а не полный текст. Поэтому важные детали лучше фиксировать в файлах проекта или в goal, а не надеяться на память чата.

4. Можно ли сменить модель на ходу?

Да, через `/model` или элемент модели в композере. Выбор применится к следующему запросу; текущий шаг доигрывает на старой модели. Reasoning effort зависит от возможностей выбранной модели: если адаптер не объявляет уровни, выбор усилия не появится.

5. Как сделать работу воспроизводимой?

Зафиксируйте версию DSH, профиль, пресет агента, модель и effort, а также журнал сессии. Модель недетерминирована, поэтому «точно тот же текст» не гарантирован, но состав инструментов, права и журнал событий воспроизводятся. Для командной работы храните конфигурацию профиля в репозитории, а секреты — вне его.

6. Можно ли редактировать историю или ветвить разговор?

Редактировать прошлые сообщения нельзя. Можно сделать Fork — копию до последнего завершённого хода — и продолжить в стороне; исходная сессия не меняется. Файлы при этом общие: fork копирует разговор, а не рабочую папку.

7. Насколько безопасен веб-поиск с точки зрения промпт-инъекций?

Контент из интернета — недоверенные данные. Модель может получить в странице инструкцию, которая попытается ей управлять. Песочница и подтверждения ограничивают ущерб, но не дают абсолютной гарантии. Не работайте в Full access с непроверенными страницами; для чувствительных задач отключайте веб или ограничивайте права.

8. Что с параллелизмом: можно ли вести несколько задач сразу?

Да, в разных сессиях. Внутри одной сессии — очередь и steer, а параллельные tool calls ограничены настройкой Agent loop. Субагенты дают дочерний параллелизм, но у них есть лимит глубины и количества. Одна сессия — один писатель: не открывайте её одновременно в двух экземплярах DSH.

9. Можно ли подключить свою модель или локальный шлюз?

Да, через Settings → Models → Add model provider → Custom model API: OpenAI Chat Completions, OpenAI Responses или Anthropic Messages. Нужен как минимум tool calling; без него агент не сможет работать полноценно. Контекстное окно и максимальный вывод задаются в каталоге модели.

10. Где хранятся секреты и попадают ли они в журнал?

Ключи провайдеров хранятся в credential store и вводятся write-only: обратно в интерфейсе не отображаются. Но всё, что вы вставили в чат или что инструмент вывел в stdout, попадает в журнал сессии. Автоматической редакции секретов нет, поэтому не вставляйте пароли и токены в сообщения.

B.3. Разработчики

1. Как расширять DSH: плагин, скилл или MCP?

Скилл — инструкции без кода, если нужно научить агента порядку действий. Плагин — код в процессе DSH: инструменты, панели, провайдеры, команды. MCP — внешний процесс или сервис, который отдаёт инструменты по протоколу. Начинайте со скилла, если не нужен код; плагин — когда нужна интеграция с API DSH; MCP — когда инструменты уже есть в другом продукте.

2. Что представляет собой минимальный плагин?

Это Cordis-плагин: npm-пакет, который экспортирует `apply(ctx)` и регистрирует свои вклады через `ctx` (инструменты, слоты, команды). Он добавляется в `dsh.profile.bundles` профиля или устанавливается через `dsh plugin add`. Точные контракты — в README соответствующих пакетов (`@deepseek-ai/dsh-tools`, `dsh-client-ui-*`),因为它们 и есть API.

3. Как добавить свой инструмент?

Через регистрацию в `ctx.tools`: имя, описание, JSON-схема аргументов, обработчик и политика доступа. Инструмент может требовать approval, иметь таймаут и возвращать структурированный результат. Дальше он автоматически попадает в каталог модели и в UI как карточка вызова.

4. Как добавить свою панель или элемент интерфейса?

Клиентские пакеты регистрируют вклады в слоты: например, `settings.section` для страницы настроек, `conversation.*` для элементов чата, `sidebar.panellist` для панели. Нужен клиентский плагин, который собирается вместе с фронтендом; серверная часть регистрирует данные и RPC.

5. Как перехватить или наблюдать работу агента?

Есть события `agent/pre-step` (можно отклонить шаг или заменить сообщения), `agent/request-error` (ретрай), `agent/turn-stopping` (продлить ход), `agent/assistant-stream` (поток). Это точка расширения для политик, аудита и интеграций, не требующая форка цикла.

6. Как тестировать плагин или профиль?

Минимально — `--dump-config` для проверки сборки дерева, затем headless-прогон как smoke-тест и профильные e2e-тесты. Для UI используйте отдельный профиль, чтобы не ломать рабочий. Проверяйте совместимость версий до публикации, а не после.

7. Как обеспечить совместимость с версиями DSH?

В манифесте плагина объявляется диапазон `engines.dsh` (или peer-диапазон DSH). Установка проверяет его и требует явного исключения для несовместимой версии. Исключение — это риск, а не решение: фиксируйте поддержку и тестируйте на целевой версии.

8. Можно ли встроить DSH в свой продукт?

Да. Есть профили `sdk` и `sdk-minimal` с JSON-RPC по stdio и `acp` для автоматизации. Они дают тот же агентский цикл, инструменты и сессии, но без Web GUI. Это профили, а не отдельные бинарники.

9. Как устроена сессия внутри?

Это event-sourced журнал: каждое модельно-видимое событие добавляется в лог, история выводится из него, проекции кэшируют состояние. Fork копирует префикс, компакция скрывает участок, но не удаляет. Поэтому сессию можно воспроизвести и выгрузить.

10. Как добавить нового провайдера моделей?

Через LLM-адаптер: маршрут, каталог моделей, схема конфигурации, маппинг reasoning. Для OpenAI/Anthropic-совместимых эндпоинтов обычно достаточно Custom model API в настройках; для нестандартного протокола нужен свой адаптер.

11. Чем отлаживать конфигурацию и загрузку плагинов?

`dsh --dump-config` показывает итоговое дерево, `--dump-default-config` — без пользовательского слоя, `--dump-config-schema` — схему. Ошибки загрузки идут в stderr с кодом; для клиентских плагинов полезен devtools/инспектор. Журнал сессии помогает понять, что реально ушло модели.

12. Можно ли переопределить системный промпт и поведение?

Да, в разумных пределах: персона пресета, файлы инструкций проекта (`AGENTS.md`), секции системного промпта, политики плагинов. Но это не «переписать модель»: базовые правила инструментов и безопасности остаются. Для устойчивых изменений лучше плагин или пресет, чем правка на каждом запросе.

B.4. DevOps, безопасность, эксплуатация

1. Как развернуть DSH на команду?

В текущей сборке это инструмент **одного пользователя**: один процесс, один токен доступа, без встроенных ролей и мультитенантности. Для команды разумны отдельные экземпляры на пользователя (Desktop или свой `DSH_HOME` + порт) либо Web за SSO-прокси, но DSH всё равно будет видеть одного пользователя. Полноценный RBAC/мультитенант — это внешняя обвязка, а не настройка DSH.

2. Как безопасно открыть Web-доступ?

По умолчанию слушается loopback. `--host` принимает только конкретный локальный IP (wildcard запрещён). Для удалённого доступа — TLS (`--tls-cert`/`--tls-key`), `--public-url` и `--trusted-host` за прокси, который снимает префикс. Порт нельзя выставлять в интернет напрямую: стартовый URL содержит токен, а cookie подписывается процессом.

3. Где хранятся данные и как их бэкапить?

В `%USERPROFILE%\.dsh`: сессии (сжатый JSONL), `workspace.json`, профили, скиллы, credential store. Бэкап — копирование каталога при остановленном процессе; иначе можно поймать недописанный журнал. Восстановление — вернуть каталог на той же или совместимой версии; форматы сессий версионированы и мигрируются. Учтите, что в каталоге есть секреты.

4. Как ограничить ресурсы?

Есть таймаут команды и лимит вывода на поток (Settings → Plugins → Shell), лимиты субагентов (глубина и параллельность), компакция контекста. Но cgroup/лимитов памяти и CPU у DSH нет: это делается средствами ОС, контейнера или виртуальной машины.

5. Насколько сильна песочница?

Это same-world confinement: процесс живёт в той же ОС и файловой системе. `workspace-write` ограничивает запись, `read-only` запрещает её, `danger-full-access` снимает ограничения. На Windows часть ограничений реализована через ACL и может иметь статус partial. Для недоверенного кода это не замена контейнеру или VM.

6. Как передавать секреты в CI?

Headless не использует вход, сохранённый в GUI: ключи берутся из переменных окружения, `.env` в текущем каталоге или `%USERPROFILE%\.dsh\.env`. Передавайте их через секреты CI, не печатайте в логах, не коммитьте. После прогона отзывайте краткоживущие токены.

7. Как встроить DSH в пайплайн?

`dsh headless "задача"` — один запуск, ответ в stdout, код 0/1. `--json` даёт поток событий, `--session-id` — продолжение. Интерактивных подтверждений в headless нет: ответчика нет, поэтому запросы подтверждения не проходят. Права и workspace должны быть ограничены заранее. Фиксируйте версию DSH и прогревайте зависимости.

8. Что с логами и аудитом?

Журнал сессии содержит промпты, вызовы инструментов и результаты; его можно выгрузить ZIP. Подтверждения и отказы пишутся в аудит сессии. Централизованного SIEM из коробки нет: есть OTel-хуки и продуктовая телеметрия Desktop, но сбор и хранение — ваша задача. Планируйте retention: автоматической очистки нет.

9. Как обновлять и откатывать?

Desktop обновляется через свой механизм обновления; CLI — переустановкой пакета. Плагины обновляются отдельно и проверяются на совместимость (`engines.dsh`). Перед обновлением: бэкап `~/.dsh`, тестовый профиль, проверка критичных плагинов. Откат — вернуть прежнюю версию приложения и каталог из бэкапа.

10. Какой сетевой доступ нужен?

API провайдера модели, веб-поиск/загрузка страниц, MCP-серверы и реестры пакетов при установке плагинов. Прокси настраивается через окружение. Встроенного allowlist доменов нет — ограничивайте egress на уровне сети/файрвола.

11. Можно ли запускать несколько экземпляров?

Да, но с разными `DSH_HOME` и портами. Общий домашний каталог на несколько процессов — риск конфликтов: одна сессия рассчитана на одного писателя, а операции с профилем сериализуются файловыми блокировками. Для параллельных CI-прогонов — эфемерный home на прогон.

12. Что делать с ошибками доступа к файлам на Windows?

Сначала проверить, что путь действительно внутри workspace и доступен пользователю. Для диагностики ACL есть скилл `diagnose-windows-sandbox-acl`: он проверяет путь и родителей и чинит подтверждённые проблемы. Не запускайте DSH от администратора «чтобы работало» — это расширяет права, а не решает причину.

B.5. Неудобные вопросы

1. Это же просто ChatGPT с файлами?

Нет. Ключевое отличие — агентский цикл: модель вызывает инструменты, получает результаты, проверяет себя и продолжает до критерия готовности. Плюс песочница, подтверждения, сессии, субагенты, цели и расписание. ChatGPT с файлами — это один запрос/ответ, здесь — управляемый процесс с доступом к среде.

2. Агент может слить мои данные?

Да, если данные попадут в контекст: они уходят провайдеру модели, а веб-инструменты и MCP могут отправлять их дальше. DSH не является DLP-системой. Меры: не вставлять секреты, ограничивать egress, использовать корпоративный шлюз, читать политику провайдера.

3. Можно ли доверить ему прод?

Нет, без ограничений и проверки. Даже с approvals модель может ошибиться, а Full access снимает тормоза. Прод — только через staging, отдельные credentials, запрет destructive-команд и обязательный human review. DSH ускоряет работу, но не заменяет процесс выпуска.

4. Почему он ошибается так уверенно?

Языковая модель оптимизирована на правдоподобный текст, а не на истину. Уверенный тон ничего не доказывает. Поэтому в DSH есть инструменты проверки: тесты, diff, чтение файлов. Правило простое: верить выводу проверки, а не формулировке.

5. Заменит ли это разработчика?

Нет. Это ускоритель для рутины: чтение кода, черновики, миграции, тесты, документация. Ответственность за архитектуру, безопасность и выпуск остаётся на человеке. Чем лучше задача декомпозирована и чем строже проверки, тем полезнее агент.

6. Почему так медленно и дорого?

Латентность складывается из reasoning, вызовов инструментов и размера контекста. Стоимость — из токенов на вход/выход и числа шагов. Помогают: более дешёвая модель, меньший effort, компакция, точные задачи без «прочитай весь репозиторий», меньше параллельных субагентов.

B.6. Если ответа нет под рукой

Не импровизируйте фактами. Рабочая формула для лектора:

> *«В этой сборке я это не проверял. Давайте посмотрим вместе: `dsh --help`, `--dump-config` или журнал сессии покажут точный ответ. Если воспроизведётся — зафиксирую и вернусь».*

Три вещи, которые стоит показать на демо: смену режима прав, остановку через Esc Esc и diff изменённых файлов. Они лучше всего объясняют, как DSH контролируется.

Руководство описывает DSH 0.2.1-rc.3. В следующих версиях названия пунктов и набор плагинов могут меняться; ориентируйтесь на подписи в интерфейсе и dsh --help.